Project Specification Document: ToolGuard

1. Application Scenarios

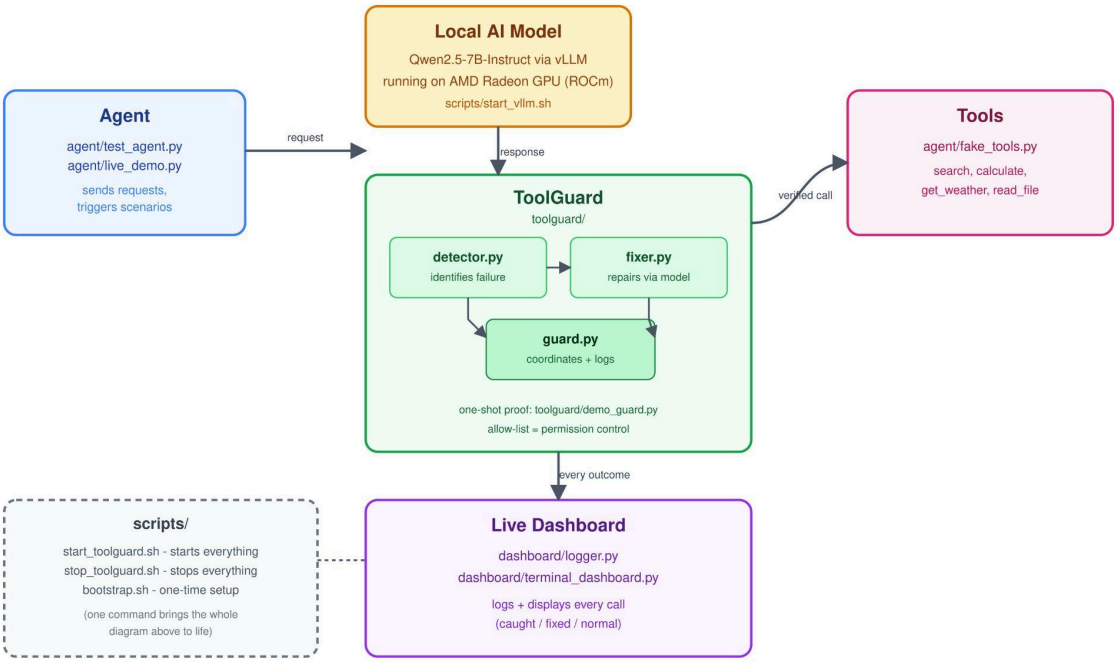
The current AI agent landscape suffers from a critical reliability gap. When an AI agent attempts to use a tool—such as performing a web search or running a calculation—the underlying model must generate a perfectly structured message. If this structure is malformed, cut off mid-stream, or leaks internal schema details, the tool call fails. In many existing frameworks, these failures are silent; the call simply vanishes without an error message, leaving the user with a stalled agent and no explanation. This project addresses this instability by providing a dedicated reliability layer that ensures tool calls are either executed correctly or repaired in real-time.

Evidence of this problem is well-documented in the developer community. We have identified public complaints where valid tool calls were silently ignored (e.g., continuedev/continue#5508) and reports of models leaking internal schema definitions rather than providing valid outputs (e.g., GLM-5.1 discussion #1). Furthermore, hardware-level issues such as multi-GPU communication errors (ROCm#6148) and device-specific bugs (ROCm#4909) persist in the ecosystem as of April 2026. These real-world issues demonstrate that the "broken tool call" is a live, ongoing pain point for developers building private AI agents.

Mainstream agent tooling has documented cases of silently dropping broken tool calls, as seen in public issue trackers like continuedev/continue#5508. Rather than letting these calls disappear, ToolGuard provides the missing developer-experience layer to make these inevitable bugs instantly recoverable.

2. Agent Architecture Diagram

The following diagram illustrates the structural relationship between the local model, the ToolGuard reliability layer, and the external tools.



The flow of information begins with the agent sending a request to the local AI model. When the model generates a response, ToolGuard intercepts it. If the response is a valid tool call, it is passed directly to the tools. If ToolGuard detects a broken call, it identifies the failure type, asks the model for a targeted repair, verifies the new output, and then executes the tool. Finally, all activity—including successes and repairs—is logged and displayed on a live terminal dashboard for the developer to monitor.

3. Core Capabilities

The team has implemented two primary agent capabilities within the ToolGuard system:

- **Tool Invocation:** The system manages the execution of functional tools. For this implementation, we include four simulated tools: search,

calculate, get_weather, and read_file. The agent interacts with these via a local model that understands the specific schemas required to trigger each action.

- **Clear Permission Control:** We have implemented a robust access-control mechanism via an explicit allow-list of tool names. This is managed within `toolguard/detector.py`. Any attempt by the model to call a tool name not present on this list is immediately rejected and flagged. This prevents the execution of "hallucinated" or unauthorized tool names, transforming a common bug into a security-first permission gate.

4. Model Introduction & Local Deployment Plan

ToolGuard utilizes the **Qwen2.5-7B-Instruct** model, chosen for its strong instruction-following capabilities. The model is deployed locally on an AMD Radeon GPU using the ROCm (Radeon Open Compute) software stack. To ensure high-performance serving, we use **vLLM**, a high-throughput inference engine. We utilize the ROCm Triton Attention backend, a specialized optimization for AMD hardware, to handle the heavy computational load of model inference.

The deployment is managed through an automated bootstrap process that installs AMD's pre-built ROCm Python environment, avoiding common compatibility issues found in standard installations.

Hardware and Memory Specifications:

- **Model Load Size:** ~14GB (Total weight size).
- **KV Cache Size:** ~27 GiB KV
- **Total GPU Memory Footprint:** ~41.3 GiB out of the **48 GiB VRAM** available on your AMD Radeon Pro GPU

The system is designed for persistence; model and compiled-graph caches are stored locally, reducing model load times from several minutes to a few seconds upon restart.

5. Inference Speed Optimization on AMD Radeon GPU

We have achieved high performance on AMD hardware by enabling specific optimizations within the vLLM framework. These include chunked prefill (setting max_num_batched_tokens=2048), which improves how the model processes initial prompts; prefix caching, which avoids redundant computations for repeated system prompts; and **ahead-of-time (AOT) CUDA graph compilation** tailored to the specific GPU architecture.

Measured Performance Figures:

- **Time to First Token (TTFT):** ~0.04 seconds (The time it takes for the model to start generating text).
- **Tokens Per Second (TPS):** ~32.8 (The speed of continuous text generation).
- **Repair Latency/Cost:** ~0.67 seconds

The ToolGuard repair mechanism is not a blind retry. When a failure is detected, the system first classifies which specific error occurred (e.g., schema leak or incomplete stream). It then constructs a targeted corrective instruction for the model, asking it to re-route or fix the call based on its understanding of intent. Finally, the system verifies that the repaired output is valid before it is ever sent to a tool.

6. Real Results

The team conducted a reliability benchmark consisting of 50 real trials against the live model. These trials included a mix of normal requests and various failure patterns (plain text responses, schema leaks, and incomplete streams) to test the system's resilience.

Success Rates:

- **Without ToolGuard:** 32% success rate
- **With ToolGuard:** 100% success rate
- **Total Improvement:** +68 percentage points

These results demonstrate that ToolGuard effectively eliminates silent failures, ensuring that even when the model makes an initial error, the agent remains functional and the developer remains informed via the live dashboard.

7. Limitations & Future Work

While the current system is highly effective, we have identified several honest limitations and areas for future development:

- **Dashboard Constraints:** We originally intended to provide a web-based dashboard; however, platform-specific port-exposure limitations on our cloud instance required us to pivot to a fully functional terminal-based dashboard.
- **Integration Depth:** The current implementation exists as a software layer. We deliberately did not attempt driver or OS-level integration, as creating a background service baked into the GPU driver is a multi-year engineering effort beyond the hackathon scope.
- **Model Optimization:** Testing the system with smaller or quantized models (compressed versions of the AI) is a logical next step to reduce hardware requirements, though these have not yet been benchmarked.

8. Team

Team Name: JINX
Track: Track 2 (Agentic AI)
Members:

- Joshua M Abraham
- Jerin Saji
- Jino J.